

Module 5 — RAG Fundamentals

Retrieval-augmented generation from ingestion to citation

Why RAG?

LLMs hallucinate. Source documents do not.

Retrieval-Augmented Generation feeds real documents into the prompt.

The model answers from evidence, and you can cite the source.

🎯 Learning goals

- Design **chunking strategies** for long-form documents and knowledge bases.
- Produce **embeddings**, store in a **vector index**, and query effectively.
- Expand a query with **alternate phrasings** or a **hypothetical passage** (HyDE).
- Build **grounded prompts** with citations back to source chunks.
- Wrap a RAG pipeline as **MCP tools** any agent can call.

The RAG pipeline

- **Ingest**: load documents, split into chunks, embed, store.
- **Retrieve**: user query → embed → vector search → top-k chunks.
- **Generate**: feed retrieved chunks + query to the LLM.
- **Cite**: link each claim in the answer back to its source chunk.



What is a vector store?

Where the embeddings live

What is a vector store?

- A database that indexes **vectors** rather than rows and columns.
- Each entry holds the embedding, the original text, and its metadata.
- You search by **distance**: hand it a vector, get the nearest ones back.
- The search is **approximate** (ANN) — a little recall traded for speed.
- Without one, answering a query means comparing it to every chunk you own.

Common vector stores

- **ChromaDB** — in-process, batteries included. What this course uses.
- **FAISS** — a library, not a server. Fast at millions of vectors, more code.
- **pgvector** — a Postgres extension. Vectors beside your relational data.
- **Qdrant**, **Weaviate** — dedicated servers with filtering and hybrid search.
- **Pinecone** — hosted and managed. You pay to have no infrastructure.



Chunking

Splitting documents so retrieval can find them

Chunking with overlap

```
def chunk_text(text, chunk_size=500, overlap=100):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]
        chunks.append({
            "text": chunk,
            "start": start,
            "end": min(end, len(text)),
            "index": len(chunks),
        })
        start += chunk_size - overlap
    return chunks
```

- → Overlap prevents sentences from being cut in half between chunks
- → Metadata (start, index) enables citation linking later

Chunking strategies

- **Fixed window**: simple, predictable size. Good baseline.
- **Sentence-aware**: split on sentence boundaries inside the window.
- **Structure-aware**: honour headings, paragraphs, or log entry boundaries.
- **Overlap**: 10-20% overlap keeps context across boundaries.
- Smaller chunks = more precise retrieval; larger = more context per hit.

Synthetic chunking

Fact extraction	FAQ extraction
Pull every atomic claim out of the passage	Generate the questions the passage answers
"Efficiency fell to 72% after the ion storm"	"What did the ion storm do to the reactor?"
A query about that claim lands on a precise vector	A crew question matches a question, not the prose
Retrieve the source passage, not the claim alone	Embed the question; hand the passage to the model



Embeddings

Turning text into something you can measure distance between

Embeddings — turning text into vectors

- An embedding maps text to a fixed-length vector in semantic space.
- Similar meanings land **close together**; unrelated ones land far apart.
- **OpenAI** `text-embedding-3-small` — fast, cheap, 1536 dimensions.
- **Sentence Transformers** — local, free, great for prototyping.
- Always embed queries with the **same model** used for documents.
- `dimensions=256` truncates the vector — smaller index, slightly worse recall.

Embed and search

```
from openai import OpenAI
import numpy as np

client = OpenAI()

def embed(texts: list[str]) -> list[list[float]]:
    resp = client.embeddings.create(
        model="text-embedding-3-small", input=texts, dimensions=256
    )
    return [d.embedding for d in resp.data]

def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def search(query, index, k=5):
    q_vec = embed([query])[0]
    scored = [(cosine_sim(q_vec, doc["vec"]), doc) for doc in index]
    scored.sort(key=lambda x: x[0], reverse=True)
    return scored[:k]
```

- → Cosine similarity: 1.0 = identical direction, 0.0 = unrelated
- → Real systems use FAISS or ChromaDB — same idea, much faster



Demo — Ingest + embed

Switch to terminal: python demo/ingest.py



Advanced techniques

What you add once the basic pipeline works

Synthetic query expansion

Query expansion	HyDE
Rewrite the question a few different ways	Write a hypothetical log entry that answers it
"reactor output after the storm"	"Efficiency fell to 72% after the ion storm…"
Search each phrasing and merge the hits	Embed that passage — it sits near real logs
Answer from the passages, not the rewrites	Throw the fiction away; cite what you retrieved

Hybrid search

- Vector search matches **meaning**. Keyword search (**BM25**) matches **tokens**.
- Ask for error code `FTL-4092` and the embedding has nothing to be similar to.
- Run both searches, then merge the two ranked lists.
- **Reciprocal Rank Fusion** scores by position: `1/(k + rank)`, k ≈ 60.
- Ranks fuse cleanly — no normalising BM25 scores against cosine distances.

↑↓ Re-ranking

- Retrieval is tuned for **recall** — cast a wide net, then order it properly.
- A **cross-encoder** reads query and passage **together**, not separately.
- That catches negation and qualification a bi-encoder averages away.
- Typical shape: retrieve 50 cheaply, re-rank 25, send the top 5 to the model.
- You pay latency per candidate — keep the candidate set small.

Metadata filtering

```
# Exact match — only engineering logs
collection.query(
    query_texts=["hull damage"],
    n_results=5,
    where={"department": "engineering"},
)

# Range — only logs since the ion storm
collection.query(
    query_texts=["hull damage"],
    n_results=5,
    where={"stardate": {"$gte": 2287.0}},
)
```

- → The filter runs first — search only ranks what survives it
- → You can only filter on metadata you stored at ingest time

Semantic caching

- The crew asks the same question twenty different ways.
- Cache on the **embedding**, not the string — "reactor status" ≈ "how is the reactor".
- On a hit above a similarity threshold, return the stored answer and skip the pipeline.
- Threshold too low and you confidently serve the answer to a different question.
- Re-indexing the corpus should drop the cache — stale answers outlive their sources.



Grounded generation

Citations and knowing when to say "I don't know"

Grounded prompt with citations

```
def build_rag_prompt(query, chunks):
    context = "\n\n".join(
        f"[{c['index']}] {c['text']}" for c in chunks
    )
    return (
        f"Answer the question using ONLY the passages below.\n"
        f"Cite passage numbers in square brackets.\n\n"
        f"Passages:\n{context}\n\n"
        f"Question: {query}"
    )

# Answer: "The outage began at 14:32 UTC [2]
#          affecting the payments service [0]."
```

- → Numbered passages let the model cite its sources
- → ONLY constrains answers to retrieved evidence



RAG as MCP tools

Make your pipeline a pluggable capability

Why wrap RAG in MCP?

- The agent calls `search_docs` or `ask_docs` — it never touches embeddings or ChromaDB.
- Any MCP-compatible agent can use your RAG pipeline without code changes.
- The server owns the index lifecycle: build once at startup, serve many queries.
- Separation of concerns — swap the vector store without touching the agent.

MCPServer tool registration

```
from mcp.server.mcpserver import MCPServer
from index_builder import load_logs, build_index, search

mcp = MCPServer("RAG Server")
logs = load_logs()
collection = build_index(logs)

@mcp.tool()
def search_docs(query: str, k: int = 5) -> str:
    """Search the document index for relevant passages."""
    hits = search(collection, query, k)
    return "\n".join(
        f"[{h['id']}] (score {h['score']:.2f}) {h['text'][:200]}"
        for h in hits
    )
```

- → Type hints generate JSON Schema — the agent discovers parameters automatically
- → The index is built once at import time and shared across all tool calls



Demo — RAG MCP server

Switch to terminal: python demo/server.py



Demo — RAG agent via MCP

Switch to terminal: python demo/agent.py



Putting it all together

Summary and exercises

Summary

Chunk for retrieval, not for reading

Smaller chunks with overlap beat whole-page dumps.

Same model for docs and queries

Embed both sides, store the vectors, and search. Mismatched models score garbage.

Ground every answer

An answer without a source is just a hallucination with confidence.

The pipeline can be tools

Wrap ingest and search as MCP tools any agent can call.

Exercises

01 — Build index: chunk, embed, and search with ChromaDB

02 — RAG chat: grounded prompts with source citations

03 — RAG MCP server: wrap the pipeline as tools for any agent



Module 5 — Complete

Next: structured facts and knowledge graphs